



การศึกษา Algorithm สำหรับการทดสอบซอฟต์แวร์ และการออกแบบ Prompt สำหรับ AI

สมาชิก

นายณัฐพงศ์ กรธนกิจ	673380038-9
นายธนกร ทองศรี	673380040-2
นายธนันชัย พันธราช	673380042-8
นายพีรพัฒน์ ป้องกันยา	673380053-3

เสนอ

ผศ.ดร.ชิตสุธา สุ่มเล็ก

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

1. อัลกอริทึมสำหรับการสร้างชุดทดสอบอัตโนมัติ

กลุ่มผู้จัดทำได้ทำการเลือกอัลกอริทึมเชิง Search-based Software Testing (SBST) โดยสองแบบที่มีหลักการต่างกันเพื่อเปรียบเทียบพฤติกรรมการค้นหาข้อมูลนำเข้าที่ทำให้เกิดความครอบคลุมของโค้ด (Code coverage) และสามารถตรวจพบข้อบกพร่อง (Fault detection) ใน Java projects ของ Defects4J ได้ ดังนี้

1.1 Differential Evolution (DE)

Differential Evolution (DE) เป็นอัลกอริทึมเชิงวิวัฒนาการ (Evolutionary Algorithm) ประเภท Population-based Metaheuristic ที่นำเสนอโดย Storn และ Price (1997) ซึ่งออกแบบมาเพื่อแก้ปัญหาการหาค่าที่เหมาะสมที่สุด (Optimization) โดยเฉพาะปัญหามิติค่าจริง (Continuous Domain) จุดเด่นของ DE คือการใช้ผลต่างระหว่างสมาชิกในประชากร (Difference Vector) เพื่อสร้าง Candidate Solution ใหม่ ทำให้เกิดการค้นหาที่สามารถปรับทิศทางตามข้อมูลของประชากรในแต่ละรอบได้ โดยมีพารามิเตอร์สำคัญ เช่น Mutation Factor (F) และ Crossover Rate (CR) ทั้งนี้ DE สามารถประยุกต์ใช้ในการสร้างชุดข้อมูลนำเข้า (Test Input) ที่ประกอบด้วยค่าตัวเลขหรือพารามิเตอร์ของเมธอด เพื่อค้นหาค่าที่สามารถกระตุ้นพฤติกรรมหรือเส้นทางการทำงานที่ต้องการทดสอบของโปรแกรมได้

ขั้นตอนการทำงานของอัลกอริทึม

1. Initialization: สุ่มสร้างประชากรเริ่มต้น (Population) โดยแต่ละสมาชิกแทนชุดข้อมูลนำเข้า (Test Input) สำหรับเมธอดที่ต้องการทดสอบ
2. Fitness Evaluation: ประเมิน Test Input ด้วย Fitness Function เช่น Branch Distance และ Approach Level เพื่อวัดความสามารถในการเข้าใกล้หรือครอบคลุม Branch เป้าหมาย
3. Mutation: สร้าง Donor Vector จากสมาชิกที่สุ่มเลือกใน Population โดยใช้สูตร

$$v_i = x_{r1} + F \times (x_{r2} - x_{r3})$$
 โดย F คือ Mutation Factor
4. Crossover: ผสม Donor Vector กับ Target Vector ตามค่า Crossover Rate (CR) เพื่อสร้าง Trial Vector ซึ่งเป็น Test Input ชุดใหม่
5. Selection: เปรียบเทียบ Fitness ของ Trial Vector กับ Target Vector และเลือกตัวที่สามารถเข้าใกล้หรือครอบคลุม Branch เป้าหมายได้ดีกว่าเข้าสู่ Population รุ่นถัดไป
6. Termination: ทำซ้ำขั้นตอน Fitness Evaluation, Mutation, Crossover และ Selection จนถึง Generation หรือ Budget ที่กำหนด หรือครอบคลุม Branch เป้าหมายได้ตามเงื่อนไข

1.2 Ant Colony Optimization (ACO)

Ant Colony Optimization เป็นอัลกอริทึมเชิง Swarm Intelligence ที่นำเสนอโดย Dorigo และคณะ ได้แรงบันดาลใจจากพฤติกรรมการหาอาหารของมดที่ใช้สารฟีโรโมน (Pheromone) เป็นสื่อกลางในการสื่อสาร ทางอ้อมระหว่างสมาชิกในฝูง เส้นทางที่มดเดินผ่านบ่อยและนำไปสู่อาหารได้เร็วจะมีปริมาณฟีโรโมนสะสมสูงขึ้น และดึงดูดมดตัวอื่นให้เลือกเดินตามเส้นทางนั้นมากขึ้นในรอบต่อไป จุดเด่นของ ACO คือเหมาะกับปัญหาที่แทน ด้วยกราฟ (Graph-based problem) ซึ่งตรงกับลักษณะของ Control Flow Graph ของโปรแกรม ทำให้อัลกอริทึม นี้เหมาะกับการค้นหาเส้นทางการทำงาน (Execution path) ของโปรแกรมเพื่อสร้างกรณีทดสอบที่ครอบคลุมสาขา ต่าง ๆ

ขั้นตอนการทำงานของอัลกอริทึม

1. Graph Construction: สร้าง Control Flow Graph (CFG) ของเมธอด โดย Node แทน Statement หรือ Basic Block และ Edge แทนเส้นทางการไหลของโปรแกรม
2. Initialization: กำหนดค่า Pheromone เริ่มต้นบนแต่ละ Edge และกำหนดค่า Heuristic เพื่อช่วยเลือกเส้นทาง เช่น ให้ความสำคัญกับ Branch ที่ยังไม่ถูกครอบคลุม
3. Path Construction: ให้ Artificial Ants เลือกเส้นทางใน CFG โดยพิจารณาจากค่า Pheromone และ Heuristic เพื่อสร้าง Execution Path
4. Input Generation: ค้นหา Test Input ที่ทำให้โปรแกรมทำงานตาม Execution Path ที่เลือก และนำไปรันเพื่อประเมิน Coverage ที่ได้
5. Pheromone Update: ลดค่า Pheromone ด้วย Evaporation และเพิ่ม Pheromone ให้กับเส้นทางที่ให้ Coverage ที่ดี เพื่อเพิ่มโอกาสถูกเลือกในรอบถัดไป
6. Termination: ทำซ้ำขั้นตอน Path Construction, Input Generation และ Pheromone Update จนถึง Iteration หรือ Budget ที่กำหนด หรือครอบคลุม Branch เป้าหมายได้ตามเงื่อนไข

2. Generative AI / AI-Assisting Tools ที่เลือก

เครื่องมือ Generative AI ที่เลือกใช้ในการสร้าง Test Suite และ Test Case ได้แก่ Claude และ Gemini โดยจะออกแบบ Prompt ให้ทั้งสองเครื่องมือได้รับข้อมูลและคำสั่งในรูปแบบเดียวกัน เพื่อให้สามารถนำผลลัพธ์มาเปรียบเทียบกันได้

2.1 Claude

Claude เป็น Generative AI ที่สามารถวิเคราะห์ Source Code และสร้างโค้ดตามคำสั่งได้ ในโครงการนี้จะใช้ Claude ในการวิเคราะห์ Java Source Code และสร้าง Unit Test ในรูปแบบ JUnit โดย Prompt จะระบุ Source Code ที่ต้องการทดสอบ เงื่อนไขในการสร้าง Test Case และกำหนดให้พยายามครอบคลุม Branch และกรณีทดสอบต่าง ๆ ให้มากที่สุด

รูปแบบ Prompt ที่ใช้

- ระบุ Java Source Code หรือ Class ที่ต้องการทดสอบ
- กำหนดให้สร้าง JUnit Test Cases
- ครอบคลุม Normal Case, Boundary Case และ Exception Case
- พยายามเพิ่ม Branch และ Code Coverage
- กำหนดให้แสดงเฉพาะ Test Code ที่สามารถนำไปใช้งานได้

2.2 Gemini

Gemini เป็น Generative AI ที่สามารถวิเคราะห์และสร้าง Source Code ได้เช่นเดียวกัน ในโครงการนี้จะใช้ Gemini สร้าง JUnit Test Cases สำหรับ Java Projects ใน Defects4J โดยใช้ Prompt รูปแบบเดียวกับ Claude เพื่อให้การทดลองและเปรียบเทียบผลของทั้งสองเครื่องมืออยู่ภายใต้เงื่อนไขเดียวกัน

รูปแบบ Prompt ที่ใช้

- ระบุ Java Source Code หรือ Class ที่ต้องการทดสอบ
- กำหนดให้สร้าง JUnit Test Cases
- ครอบคลุม Normal Case, Boundary Case และ Exception Case
- พยายามเพิ่ม Branch และ Code Coverage
- กำหนดให้แสดงเฉพาะ Test Code ที่สามารถนำไปใช้งานได้

3.พรอมพ์ที่ออกแบบไว้สำหรับสร้างชุดทดสอบ

เพื่อให้สามารถเปรียบเทียบประสิทธิภาพของ Claude และ Gemini ได้อย่างเป็นธรรม จะใช้ Prompt รูปแบบเดียวกันกับทั้งสองเครื่องมือ โดยเปลี่ยน Source Code ตาม Java Project และ Class ที่นำมาทดสอบ

3.1 Prompt สำหรับ Claude

คุณเป็นผู้เชี่ยวชาญด้านการทดสอบซอฟต์แวร์ ให้วิเคราะห์ Source Code ภาษา Java จากโปรเจกต์ใน Defects4J ที่กำหนด และสร้างกรณีทดสอบระดับ Unit Test ด้วย JUnit

ข้อมูลโปรเจกต์

- Project: [ชื่อโปรเจกต์]
- Project ID/Version: [หมายเลขที่ทดสอบ]
- Class ที่ต้องการทดสอบ: [ชื่อ Class]

ข้อกำหนด

1. สร้าง Test Case สำหรับ Java Class ที่กำหนด
2. ครอบคลุม Normal Case, Boundary Case และ Exception Case หากมี
3. พยายามเพิ่ม Branch Coverage และ Code Coverage ให้มากที่สุด
4. สร้าง Test Case ที่มีโอกาสตรวจพบข้อบกพร่องของโปรแกรม
5. Test ที่สร้างต้องสามารถ Compile และ Execute กับโปรเจกต์ที่กำหนดได้
6. แสดงผลลัพธ์เฉพาะโค้ด JUnit Test เท่านั้น

Java Source Code:

[ใส่ JAVA SOURCE CODE ที่นี่]

3.2 Prompt สำหรับ Gemini

คุณเป็นผู้เชี่ยวชาญด้านการทดสอบซอฟต์แวร์ ให้วิเคราะห์ Source Code ภาษา Java จากโปรเจกต์ใน Defects4J ที่กำหนด และสร้างกรณีทดสอบระดับ Unit Test ด้วย JUnit

ข้อมูลโปรเจกต์

- Project: [ชื่อโปรเจกต์]

- Project ID/Version: [หมายเลขที่ทดสอบ]
- Class ที่ต้องการทดสอบ: [ชื่อ Class]

ข้อกำหนด

1. สร้าง Test Case สำหรับ Java Class ที่กำหนด
2. ครอบคลุม Normal Case, Boundary Case และ Exception Case หากมี
3. พยายามเพิ่ม Branch Coverage และ Code Coverage ให้มากที่สุด
4. สร้าง Test Case ที่มีโอกาสตรวจพบข้อบกพร่องของโปรแกรม
5. Test ที่สร้างต้องสามารถ Compile และ Execute กับโปรเจกต์ที่กำหนดได้
6. แสดงผลลัพธ์เฉพาะโค้ด JUnit Test เท่านั้น

Java Source Code:

[ใส่ JAVA SOURCE CODE ที่นี่]

เอกสารอ้างอิง

- Price, K. V. (2013). Differential evolution. In I. Zelinka, V. Snášel, & A. Abraham (Eds.), *Handbook of optimization* (Vol. 38). Springer. https://doi.org/10.1007/978-3-642-30504-7_8
- S, S. S., & S S, V. C. (2020). An ant colony optimization algorithm based automated generation of software test cases. In Y. Tan, Y. Shi, & M. Tuba (Eds.), *Advances in swarm intelligence* (Vol. 12145). Springer. https://doi.org/10.1007/978-3-030-53956-6_21
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (pp. 437–440). Association for Computing Machinery. <https://doi.org/10.1145/2610384.2628055>